

Contents

1 Implementation Plan: Unify interior and shared fault-QP computation on the ADER sub-step path	1
1.1 Summary — read this first	1
1.2 How to read this plan	2
1.3 Glossary	2
1.4 Technical Overview	3
1.5 Constraints	5
1.6 Phase 0: Reproduce the divergence with a failing test — no behavior change	5
1.7 Phase 1: One frame rule for both face classes	7
1.8 Phase 2: Shared faces consume the sub-step buffer — retire the R-1601 fallback	9
1.9 Phase 3: Demote the boss-broadcast from a corrector to an assertion	10
1.10 Phase 4: Forced rupture uses one clock	11
1.11 Phase 5: Collapse the duplicated interior/shared fault code into one routine	12
1.12 Phase 6: Revalidate and regenerate multi-rank references	13
1.13 Testing Strategy	14
1.14 Risk Assessment	14
1.15 Appendix A — Evidence index	16

1 Implementation Plan: Unify interior and shared fault-QP computation on the ADER sub-step path

- **Date:** 2026-07-09
- **Author:** Claude (code-explore + code-plan)
- **Status:** PLAN ONLY — no code changes implemented.
- **Supersedes the mitigation in:** `dynamic/wave_operator.inl` R-1601 shared-fault fallback (cited and justified in Phase 2; per CLAUDE.md this fix is NOT reverted blindly).
- **Debug history read:** `debug_document/tpv104_debug_document/SUBSTEP_ITERATOR_MPI_FIX.md`, `.../SUBSTEP_NP_GT_1_HANG_REVIEW.md`, `.../ADER_ITERATOR_SYM1000_RESULTS.md`.

1.1 Summary – read this first

The problem. A fault quadrature point that happens to land on an MPI partition boundary is computed by different code than an identical point in the middle of a rank. The interior point is advanced with several small sub-steps. The boundary point is advanced with one big step covering the whole time step. Worse, the boundary point’s friction is evaluated using slip that has *already* been advanced to the end of the step, but a clock reading from the *start* of the step. This means the physical answer depends on how the mesh was cut across processors, which it must not. The effect is present for every friction law, and it silently grows with the number of ranks.

The root cause. For interior faces the code defines “which side of the fault is the plus side” using a robust geometric test. For boundary faces it uses a different, floating-point-noisy test derived from

the mesh’s per-point normal. The second test was already known to be unreliable — it was replaced everywhere *except* on boundary faces. When the two disagree, the fault normal ends up pointing into the plus side instead of away from it, the friction traction flips sign, and the simulation blows up. That blow-up is exactly why a previous fix disabled sub-stepping on boundary faces rather than solving the underlying problem.

The fix. Make boundary faces use the same robust geometric side-test that interior faces already use. Once the two agree, boundary faces can safely run the identical sub-step loop as interior faces, and the special-case fallback is deleted. Finally, fold the two near-duplicate code paths into one routine so they cannot drift apart again.

Expected outcome. Every fault point is computed by the same code with the same arguments, so the answer no longer depends on the partition. On the reference mesh at 10 ranks, roughly **5% of fault points** are currently on partition boundaries and get the degraded treatment; at 1 rank the figure is **0%**, which is why no single-rank test has ever caught this. The accuracy of those 5% improves from one-shot to sub-stepped, and one redundant friction solve per boundary point per step is eliminated. This is a **correctness and reproducibility** fix, not a performance fix; the speed gain is negligible.

Main tradeoff / biggest risk. Turning sub-stepping back on for boundary faces is precisely the change that previously produced a numerical explosion. If the side-test fix is wrong or incomplete, the explosion returns. The plan therefore lands the side-test fix and its cross-rank assertion *before* re-enabling anything, and requires a failing-then-passing test at 2 ranks first.

What this does NOT do. It does not change any single-rank result — single-rank runs have no partition boundaries, so they are bit-for-bit unchanged. It does not change the sub-step scheme itself, the friction laws, or the time integrator. It will, however, **change multi-rank results** for TPV102, TPV104, TPV205 and SAFS, because those results are currently partition-dependent. Those reference traces must be regenerated, and that needs explicit sign-off.

1.2 How to read this plan

- The **Summary** above is the whole idea. If you only read one section, read that.
- Each **Phase** opens with a one-sentence goal — skim those to see the shape of the work.
- The **Detailed Requirements** under each phase are the contract for the implementation agent.
- The **Glossary** defines every shorthand used below.

1.3 Glossary

Label	Plain-language meaning
interior fault face	A fault face whose two elements both live on the same MPI rank.
shared fault face	A fault face on a partition boundary; its second element lives on another rank.
QP	Quadrature point on a fault face. Each face carries <code>nb_f_per_face</code> of them.
the side-routing bit	<code>elem1_on_plus</code> — per-face, geometric, FP-robust. True iff <i>this rank’s</i> <code>Elem1</code> sits on the canonical “+” side. It is rank-local : on a shared face the two ranks hold opposite values, because each rank’s <code>Elem1</code> is its own element.

Label	Plain-language meaning
the frame bit	The bit that decides whether the canonical fault normal is +normal or —normal. Today it is !elem1_on_plus on interior faces and qpd.sign_flipped on shared faces. This inconsistency is the bug.
sign_flipped	A per-QP bit derived from MFEM's CalcOrtho normal. fault_basis.hpp:33 documents it as “ Diagnostic only; sign already baked into basis vectors ” — yet the three shared-fault sites use it to negate those very basis vectors. It is also documented as FP-bimodal across the QPs of one face on y-mirror meshes (wave_operator.hpp:835); R-101 replaced it with the side-routing bit on interior faces only.
the sub-step buffer	substep_I_imp_{plus,minus}_flat_ — the friction iterator's per-QP time-integrated imposed states, published to the wave operator via SetSubStepFaultImposedStates.
the R-1601 fallback	The current code in ComputeADERSharedFaceFluxRHS that discards the sub-step buffer for shared QPs and re-solves them inline over the full macro dt.
the boss-broadcast one-shot solve	The per-step MPI reconcile (wave_operator.inl:5686–5749) that overwrites the higher-rank's DOFData for a shared QP with the lower-rank's copy. A single friction solve over the whole macro dt, on ADER time-integrated states.
sub-step solve	0 friction solves at the ADER sub-step nodes, with slip and state evolving between them.

1.4 Technical Overview

On the production path (`--time-integrator ader, fault_iterator = "substep"`), the driver runs `AdvanceADERWithSubStep_Spatial` (`drivers/spatial_dyn_driver.cpp:399–515`): it builds point-wise sub-step bulk states at every fault QP (interior **and** shared, via a per-sub-step `ExchangeFaceNbrData` at `wave_operator.inl:2318–2330`), runs `iterator.Advance(...)` over **all** `num_fault_total` QPs, publishes the sub-step buffer, then runs the ADER corrector.

`ComputeADERFaceFluxRHS` (interior) **consumes** that buffer (`wave_operator.inl:4288–4327`). `ComputeADERSharedFaceFluxRHS` (shared) **discards** it — literally `(void)substep_I_imp_plus_flat_;` at `wave_operator.inl:5340–5342` — and re-solves inline over the full macro dt (`:5314–5339`). Since the iterator already ran and already advanced `slip1/slip2` and `psi` for those same shared QPs, the inline solve consumes **end-of-step slip** with a **start-of-step clock** (`GetTime()`), then the boss-broadcast makes the two ranks agree on the result. The blocker to simply consuming the buffer is the frame bit (Glossary), which differs between interior and shared faces.

Key files: `dynamic/wave_operator.inl` (six frame-bit sites, two corrector branches, the boss-broadcast), `dynamic/wave_operator.hpp` (accessors, stale docstring at `:555–562`), `dynamic/friction_substep_iterator` (the iterator; already loops over all QPs), `drivers/spatial_dyn_driver.cpp` (`AdvanceADERWithSubStep_Spatial`).

The defect, stated precisely

Everything below is **verified from the source**, and is stated without assuming any particular absolute sign convention for the Godunov normal.

Fact 1. FaultBasisQPData stores basis vectors with the sign **already applied**, and marks the flip bit as diagnostic (fault/fault_basis.hpp:30–33):

```
real_t normal[3];    ///< Unit normal (with sign baked in)
real_t tangent1[3];  ///< dip    (with sign baked in)
real_t tangent2[3];  ///< strike(with sign baked in)
bool   sign_flipped; ///< Diagnostic only; sign already baked into basis vectors
```

It is set once as `sign_flipped = NormalNeedsFlipToCanonical(n_raw, ref_normal, dim)` (fault_basis.hpp:427), i.e. it is a **per-QP** bit derived from the CalcOrtho normal.

Fact 2. The three **interior** fault sites build the canonical frame from the per-face, geometric side-routing bit:

```
const bool should_negate_frame = !elem1_on_plus;    // :2221, :2972, :4217
can_n[d] = should_negate_frame ? -qpd.normal[d] : qpd.normal[d];
```

Fact 3. The three **shared** fault sites build it from the diagnostic-only bit instead:

```
can_n[d] = qpd.sign_flipped ? -qpd.normal[d] : qpd.normal[d];    // :2513, :3646, :5252
```

The defect. The same quantity — the canonical fault frame — is computed by **two different rules**, and the shared rule *re-applies* a flip to basis vectors whose sign the header says is already baked in, using a field the header calls diagnostic-only. The two rules agree only if `sign_flipped == !elem1_on_plus` at every QP. They cannot be relied on to agree, because `elem1_on_plus` is **per-face and geometric** while `sign_flipped` is **per-QP and derived from a floating-point CalcOrtho normal** — documented as *bimodal across QPs of one face* on y-mirror meshes (wave_operator.hpp:835, the R-101 rationale). When they disagree, adjacent QPs on the same shared face receive **opposite canonical frames**, which flips the sign of the traction fed to the friction solve. An antiparallel traction is the textbook positive-feedback blow-up in this code base (CLAUDE.md, “Slip rate direction ... Antiparallel sign creates positive feedback → unbounded growth”).

That is consistent with the recorded failure: $\tau \approx 4e28$ within ~ 7 macro steps at `np=10` on `tpv104/mesh/tpv104_symmirror_1000m.msh` — **a y-mirror mesh by construction** (ADER_ITERATOR_SYM1000_RESUL) i.e. exactly the geometry where `sign_flipped` is bimodal.

The fix, stated conservatively. We do not need to know which absolute convention is “correct” in the abstract. The interior rule is the one that is exercised at `np=1` by every regression in the suite, and `np=1` is the correctness oracle (§Testing Strategy). Therefore: **make the shared sites use the interior rule**. After that, one rule computes the frame everywhere, and Phase 5 makes it physically impossible for the two to diverge again.

What is NOT yet established (Phase 1 must settle it, do not assume). 1. Whether `!elem1_on_plus` yields a *rank-independent* canonical frame on a shared face. The construction comment at `wave_operator.inl:495–508` states that the canonicalized normal used to derive `elem1_on_plus` is rank-independent, and that exactly one of the two ranks

gets `elem1_on_plus == true`. That makes rank-independence plausible but not proven.
2. The absolute orientation contract of `GodunovFlux::BuildRotationInverse(can_n, ...)` — i.e. whether `can_n` is required to point plus→minus, `Elem1→Elem2`, or neither.

Phase 1 pins both by direct assertion **before** Phase 2 depends on them. If (1) is false, **STOP and re-plan** — do not proceed to Phase 2.

1.5 Constraints

- **Do not blind-revert R-1601.** CLAUDE.md forbids reverting a prior fix without citing the debug document and showing evidence. Phase 2 cites `SUBSTEP_ITERATOR_MPI_FIX.md` + the R-1601 comment and only re-enables *after* Phase 1 removes the cause.
 - **np=1 must stay bit-exact.** A serial run has zero shared fault faces, so every shared-face code path is unreachable. Any change that alters an `np=1` trace is a bug in the change.
 - **sign_flipped may not simply be deleted.** It is still the frame bit for non-fault faces and is consumed elsewhere; only the six fault-face sites listed below are in scope.
 - **Fault-local frame convention is fixed** (CLAUDE.md): `tangent1 = dip`, `tangent2 = strike`; `DOFData` channel 1 = dip, channel 2 = strike. Do not touch.
 - **sigma_n > 0 is compression**; slip-rate is parallel to traction. Any sign change must be justified against these, not against “it made the test pass”.
 - **Multi-rank reference traces will change.** `TPV102/104/205` and `SAFS np>1` gold outputs are currently partition-dependent and will move. Regeneration requires user sign-off (Phase 6).
 - **No Frontera runs without explicit approval** (project memory). Local MPI is limited to `np ≤ 10`.
-

1.6 Phase 0: Reproduce the divergence with a failing test – no behavior change

In one sentence: we can demonstrate, in a test that runs in seconds, that the same physical fault point gives a different answer depending on whether it lands on a partition boundary.

Goal

Author the `np=2` shared-fault parity test that `SUBSTEP_ITERATOR_MPI_FIX.md` explicitly lists as never-written and “merge-blocking”, and use it to capture the current (wrong) behavior as RED. Nothing in `dynamic/` or `drivers/` changes in this phase.

Files to Create

- `tests/parallel/test_shared_fault_substep_parity_np2.cpp` — the RED test.

Files to Modify

- `Makefile` — add `seas_test_shared_fault_substep_parity_np2` (4-part idiom: `SRC/OBJ` vars, `obj` rule, `link + test-shared-fault-substep-parity-np2` run target, register in `SEQ_MINIAPPS` and in the parallel-test aggregate). Mirror `tests/parallel/test_bimaterial_seam_fault_np2`

Detailed Requirements

1. **Fixture.** Build a small box ParMesh containing a single planar fault whose faces are forced across the partition, mirroring tests/parallel/test_bimaterial_seam_fault_np2.cpp. Assert `wave.GetNumSharedFaultQPs() > 0` at `np=2` — the fixture is worthless otherwise. Assert `wave.GetNumSharedFaultQPs() == 0` at `np=1`.
2. **Comparison.** Run one macro step of `AdvanceADERWithSubStep_Spatial` at `np=1` and at `np=2` on the identical physical mesh and identical initial `Q`, with LSW friction, `ader_order = 3`. Gather `DOFData` for the fault QPs by physical coordinate (use `MakeFaceKey`, `dynamic/shared_fault_key.hpp`) and compare `slip1`, `slip2`, `V1`, `V2`, `tau1_corr`, `tau2_corr`, `sigma_n_corr` at matching QPs.
3. **Expected RED result.** QPs that are interior at `np=1` but shared at `np=2` must differ by more than round-off. Record the observed max relative difference in the test's output; that number is the Phase-2 acceptance target (it must fall to round-off).
4. **A second RED assertion, the (INV) violation.** Add a standalone check that, for every shared fault QP, the canonical normal points from plus to minus:

```
// canonical normal must point plus -> minus: dot(can_n, c_plus - c_minus) < 0
bool CanonicalNormalPointsPlusToMinus(const real_t can_n[3],
                                       const real_t centroid_plus[3],
                                       const real_t centroid_minus[3]);
```

Evaluate it with today's shared frame bit (`qpd.sign_flipped`) on a y-mirror fixture. Expect it to FAIL for at least one QP. This is the direct evidence for the Phase-1 fix.

5. **Do not “fix” anything here.** The test is committed RED-but-skipped (guarded by an env var `SEAS_EXPECT_SHARED_FAULT_PARITY=1`) so make `test` stays green until Phase 2 lands.

Edge Cases to Handle

- A rank with **zero** fault faces must not deadlock — the per-sub-step `ExchangeFaceNbrData` is a collective. This is the R-1600 defect; tests/parallel/test_R1600_substep_no_fault_rank_no_hang.cpp already covers it. Reuse, do not re-derive.
- `np=2` partitions are METIS-dependent. Force the partition explicitly rather than hoping.

Acceptance Criteria

- ☐ At `np=2` the fixture reports `GetNumSharedFaultQPs() > 0`.
- ☐ The `np=1` vs `np=2` comparison FAILS today (this is the point), with the max relative difference recorded in the test log.
- ☐ The (INV) check FAILS for ≥ 1 shared QP on the y-mirror fixture with `sign_flipped`.
- ☐ `make test` remains green (new test is env-gated off by default).

Dependencies

- Depends on: nothing. Required by: every later phase (it is the oracle).

1.7 Phase 1: One frame rule for both face classes

In one sentence: the canonical fault frame on boundary faces is now built by the same robust geometric rule that interior faces already use, instead of a bit the header calls diagnostic-only.

Goal

Replace the per-QP, FP-derived `qpd.sign_flipped` frame bit with the per-face, geometric `!elem1_on_plus` at the three shared-fault sites. This is R-101 applied to the shared path, which it was never applied to. **First**, pin the frame contract that the interior rule encodes, so the change is provable rather than merely plausible.

Bit-neutrality note (why this is lower risk than it looks). On any mesh where `sign_flipped == !elem1_on_plus` at every fault QP — which is every non-degenerate mesh, since both bits then encode the same geometric fact — the three rewritten expressions produce *identical* values, so Phase 1 is bit-exact even at $np > 1$. Only FP-degenerate faces (the y-mirror case, where `sign_flipped` is documented bimodal) change behavior. Phase 1 must therefore ship with a diagnostic that COUNTS the QPs where the two bits disagree, so we know exactly which meshes are affected before anything else is touched.

Files to Modify

- `dynamic/wave_operator.inl` — three sites:
 - :2513 in `EvaluateBulkAtFaultQPsCanonical` (shared loop)
 - :3646 in `ComputeSharedFaceFluxRHS` (RK path)
 - :5252 in `ComputeADERSharedFaceFluxRHS` (ADER path)
- `dynamic/wave_operator.hpp` — add the cross-rank assertion declared below; correct the stale docstring at :555–562 (it claims both branches consume the sub-step buffer; they do not).

Detailed Requirements

0. **Pin the frame contract before changing anything.** Read `GodunovFlux::BuildRotationInverse` and its callers, and write down, in a comment block at the top of the fault section of `wave_operator.inl`, what `can_n` is required to be: does it point `plus→minus`, `Elem1→Elem2`, or is only the (frame, routing) *pair* constrained? Then add a `MFEM_ASSERT` on **interior** faces that the stated contract holds. If the assertion fails on interior faces, the contract as written is wrong — **STOP and re-plan**; do not touch the shared sites. This step exists because `qpd.normal` has its *sign already baked in* (`fault_basis.hpp:30–33`), which makes the meaning of a further negation non-obvious.
1. **Disagreement census (ship this first, on its own).** Add an env-gated diagnostic (`SEAS_DIAG_FAULT_FRAME_BITS=1`) that, for every fault QP (interior and shared), reports `count(sign_flipped != !elem1_on_plus)` and the face centroids where they differ. Run it on: the TPV26 smoke mesh, `tpv205/mesh/tpv2053d_200m.msh`, and `tpv104/mesh/tpv104_symmirror_1000m.msh` at $np \in \{1, 2, 10\}$. **Expected:** zero disagreements on the non-mirror meshes, non-zero on `symmirror`. This both confirms the diagnosis and tells us the exact blast radius before any behavior changes.
2. At each of the three sites, replace

```
can_n[d] = qpd.sign_flipped ? -qpd.normal[d] : qpd.normal[d]; // and t1, t2
```

with the interior rule already used at :2221, :2972, :4217:

```
const bool should_negate_frame = !elem1_on_plus; // R-101, now applied to shared faces
can_n[d] = should_negate_frame ? -qpd.normal[d] : qpd.normal[d]; // and t1, t2
```

elem1_on_plus at the shared sites is shared_fault_elem1_on_plus[sf_idx], already in scope.

3. Verify the orientation hypothesis before relying on it. Add

```
/// Asserts, for every shared fault face, that (a) exactly one of the two owning
/// ranks has elem1_on_plus == true, and (b) both ranks reconstruct the SAME
/// canonical normal can_n (to `tol`). Aborts on violation. No-op at np == 1.
void VerifySharedFaultCanonicalFrame(real_t tol = 1e-12) const;
```

in dynamic/wave_operator.{hpp,inl}. Implement by gathering (FaceVertexKey, elem1_on_plus, can_n[3]) per shared fault QP and pairing with MakeFaceKey (dynamic/shared_fault_key.hpp), reusing the gather in ExchangeAndPairSharedFaultQPs (wave_operator.inl:6209-6231). Call it once from the driver after operator construction, behind MFEM_DEBUG or an env gate so production cost is zero.

4. **(INV) assertion.** In the same routine, assert $\text{dot}(\text{can_n}, \text{centroid_plus} - \text{centroid_minus}) < 0$ for every shared fault QP.
5. Do **not** delete qpd.sign_flipped or the FaultBasisQPData field. It remains the frame bit for non-fault shared faces (:3646 is inside a fault branch; leave the non-fault branch alone).

Interfaces

- `void WaveOperator<MeshType>::VerifySharedFaultCanonicalFrame(real_t tol = 1e-12) const;`
- `bool CanonicalNormalPointsPlusToMinus(const real_t can_n[3], const real_t c_plus[3], const real_t c_minus[3]);` (free function, header-only, shared with the Phase-0 test).

Edge Cases to Handle

- **np = 1:** no shared fault faces $\Rightarrow$ all three sites are unreachable $\Rightarrow$ bit-exact by construction. Assert this in the test, do not assume it.
- **A rank owning zero shared fault faces** must still enter the collective in `VerifySharedFaultCanonicalFrame` (R-1600 lesson: gate on *total* shared faces, never on fault-only shared faces).
- **Degenerate elem1_proj == face_proj** (element centroid exactly on the face plane). Today :521 uses a strict $<$. Assert a minimum separation margin; abort with the face centroid on violation rather than silently picking a side.

Acceptance Criteria

- ☐ `VerifySharedFaultCanonicalFrame()` passes at np=2, 4, 8 on the Phase-0 fixture.
- ☐ The Phase-0 (INV) check now PASSES for every shared QP (it failed in Phase 0).
- ☐ `TamperSharedFaultElem1OnPlus(idx)` (the existing test-only mutator, wave_operator.hpp:849)

makes `VerifySharedFaultCanonicalFrame()` abort — proving the assertion has teeth.

- `np=1`: every existing trace is **bit-identical** (`test_friction_substep_iterator_parity`, TPV205/102/104 regressions).
- `tpv104_symmirror_1000m.msh` at `np=10` runs 20 macro steps with $\max|Q| < 1e12$ (i.e. no 4e28 overflow) **with the R-1601 fallback still in place** — this phase must not change shared-face flux yet, only the frame it is computed in.

Dependencies

- Depends on: Phase 0. Required by: Phase 2 (Phase 2 is unsafe without this).

1.8 Phase 2: Shared faces consume the sub-step buffer – retire the R-1601 fallback

In one sentence: boundary fault points are now advanced by the same sub-step loop as interior points, instead of a single big-step solve.

Goal

Delete the R-1601 special case so `ComputeADERSharedFaceFluxRHS` consumes `substep_I_imp_{plus,minus}_flat_` exactly as the interior branch does.

Justification for superseding R-1601 (CLAUDE.md requires this). R-1601 (`wave_operator.inl:5285–5301`, `ADER_ITERATOR_SYM1000_RESULTS.md:23`) disabled sub-stepping on shared QPs because unifying them produced $\tau \approx 4e28$ at `np=10` on a y-mirror mesh. Its own comment names the blocker: “*until the iterator’s per-shared-QP physics is reconciled with the corrector’s frame convention*”. Phase 1 performs exactly that reconciliation and proves it with `VerifySharedFaultCanonicalFrame` + the (INV) assertion. R-1601 is therefore a **mitigation whose precondition Phase 1 removes**, not a fix being discarded. The Phase-1 acceptance criterion (“`symmirror np=10`, 20 steps, no overflow, fallback still in place”) isolates the frame fix from the re-enable, so if the overflow returns in Phase 2 we know it is the buffer, not the frame.

Files to Modify

- `dynamic/wave_operator.inl:5296–5342` — replace the unconditional inline dispatch with the same gate the interior branch uses at `:4288–4301`; keep the inline `EvaluateADER*` calls as the else branch (reached only when no buffer is installed, i.e. the ADER one-shot path).
- `dynamic/wave_operator.hpp:555–562` — the docstring becomes true again; keep it in sync.

Detailed Requirements

1. Insert, before the inline dispatch:

```
if (substep_I_imp_plus_flat_ != nullptr && substep_I_imp_minus_flat_ != nullptr &&
    dof_idx >= 0 && dof_idx < substep_n_total_fault_qps_)
{
    const real_t *src_p = substep_I_imp_plus_flat_ + dof_idx * NUM_STATE;
    const real_t *src_m = substep_I_imp_minus_flat_ + dof_idx * NUM_STATE;
    for (int c = 0; c < NUM_STATE; c++) { I_imp_plus[c] = src_p[c]; I_imp_minus[c] = src_m[c];
    // v_imp recovery for the TPV6/7 station writer, mirroring :4315–4326
```

```

}
else { /* existing inline EvaluateADER* dispatch, unchanged */ }

```

dof_idx on the shared branch is already absolute (shared_fault_dof_offset[sf] + q, per R-1304), so the same gate expression is correct.

2. Delete the three (void)substep_I_imp_* discards at :5340–5342.
3. Port the v_imp_plus/v_imp_minus recovery block (:4315–4326) to the shared branch. Without it the TPV6/7 station writer reads zero velocity on seam stations, exactly as the interior comment warns.
4. **Do not** remove VerifyForcedRuptureTimeReady yet; Phase 4 does that.

Edge Cases to Handle

- **ADER one-shot** (fault_iterator != "substep"): buffer pointers are null $\Rightarrow$ else branch $\Rightarrow$ behavior unchanged, on both interior and shared. Assert with a one-shot regression run.
- **RK path** untouched — ComputeSharedFaceFluxRHS never had a buffer gate and does not get one.
- **A shared QP whose dof_idx exceeds substep_n_total_fault_qps_** must fall to the inline branch rather than read out of bounds. The gate already handles this; add an MFEM_ASSERT.

Acceptance Criteria

- ☐ Phase-0's np=1 vs np=2 parity test now PASSES to round-off (flip its env gate on by default).
- ☐ tpv104_symmirror_1000m.msh, np=10, ader_order $\in \{2, 3, 4\}$, 2.0 s: no overflow; hypocenter dip-slip drift is $\leq$ the one-shot value reported in ADER_ITERATOR_SYM1000_RESULTS.md.
- ☐ np=1 traces bit-identical (again — the gate must be unreachable at np=1).
- ☐ TPV205 spatial: np=1 vs np=4 V_max_global agree to $\leq 1e-10$ relative at t = 0.05 s.
- ☐ One inline EvaluateADER* call per shared QP per macro step is eliminated (verify by counter or profiler; expect a small, positive speedup).

Dependencies

- Depends on: Phase 1. Required by: Phases 3, 4, 5.

1.9 Phase 3: Demote the boss-broadcast from a corrector to an assertion

In one sentence: the two ranks that share a fault point now agree because they computed the same thing, not because one overwrote the other.

Goal

The boss-broadcast (wave_operator.inl:5686–5749) currently *masks* cross-rank disagreement by copying the lower rank's DOFData over the higher rank's. Once Phase 1 + 2 make both ranks run identical canonical physics on identical canonical inputs, disagreement should be round-off only. Keep a deterministic reduction (bitwise reproducibility across rank counts is still desirable), but add an assertion so real divergence can never hide again.

Files to Modify

- `dynamic/wave_operator.inl:5686-5749` — keep the exchange, add a pre-overwrite tolerance check.

Detailed Requirements

1. Before overwriting, compute the max relative difference between the local and peer payload.
2. If it exceeds `shared_fault_reconcile_tol_` (new member, default `1e-10`, settable), MFEM_ABORT with the face centroid, the offending field name, and both values.
3. Keep the boss-wins overwrite **after** the check. It still buys bitwise determinism against FP-noise in the neighbor interpolation; it must no longer buy correctness.
4. Expose `void SetSharedFaultReconcileTol(real_t)` so the Phase-0 test can tighten it.

Edge Cases to Handle

- The payload includes `slip_rate` and the canonical `I_imp (:5703-5719)`. `I_imp` is now the iterator's output; compare it too — a mismatch there is the direct signature of a frame bug.
- Early steps where every field is exactly `0.0`: use an absolute floor in the relative test.

Acceptance Criteria

- ☐ At `np=2/4/8` on the Phase-0 fixture the max relative disagreement is $< 1e-12$ for 100 steps.
- ☐ Deliberately reverting one Phase-1 site makes this assertion fire (negative control).
- ☐ `tests/parallel/test_shared_fault_reconcile_cross_rank.cpp` still passes.

Dependencies

- Depends on: Phase 2. Required by: nothing (independent hardening).
-

1.10 Phase 4: Forced rupture uses one clock

In one sentence: the friction coefficient on a boundary point is evaluated at the same instant as on an interior point, so the forced-rupture front no longer lags across partition seams.

Goal

This dissolves the open R-002 finding from `tpv26/REVIEW_tpv26_phase2_3_2026-07-09.md`. Once shared QPs consume the iterator's buffer, their $\mu(\delta, t)$ is the iterator's sub-step-time value, and `GetTime()` is no longer read on the fault path.

Files to Modify

- `dynamic/wave_operator.inl` — `EvaluateADER_LSW_ForcedRupture` is now called only on the ADER one-shot path; keep `VerifyForcedRuptureTimeReady` there, delete the shared-branch call.
- `dynamic/friction_substep_iterator.hpp` — remove the R-002 caveat from the `WaveOpLaw()` doc.
- `drivers/spatial_dyn_driver.cpp` — remove the `t0_s > 0 && nprocs > 1` guard added as the R-002 Option-A mitigation. It is no longer needed. Keep the `t0_s >= 0` parser check.
- `tests/unit/test_forced_rupture_iterator_parity.cpp` — retire `Test_R002_Seam_Interior_Mu_Lag_Bound` replace with an equality test.

- `tpv26/REVIEW_tpv26_phase2_3_2026-07-09.md` — mark R-002 RESOLVED, citing this plan.

Detailed Requirements

1. Assert that on the ADER sub-step path the fault never reads `GetTime()`: `grep` must show zero `GetTime()` uses inside fault branches of `ComputeADER{,Shared}FaceFluxRHS` when the buffer gate is taken.
2. Replace the bounded-lag test with: seam and interior QPs at equal $(\delta, T_{\text{forced}}, t_0)$ produce **bit-identical** μ . Since both now come from the same `StepOneQP_` call, this is exact.

Acceptance Criteria

- ☐ `test_forced_rupture_iterator_parity` asserts exact seam/interior μ equality (tol 0.0).
- ☐ A forced-rupture run on a fault forced across a seam produces `np=1` vs `np=2` rupture times agreeing to $\leq 1e-12$ s.
- ☐ R-002 is marked RESOLVED with evidence.

Dependencies

- Depends on: Phase 2. Required by: nothing.

1.11 Phase 5: Collapse the duplicated interior/shared fault code into one routine

In one sentence: there is now a single function that computes a fault QP, so interior and shared can never drift apart again.

Goal

The two corrector branches are $\sim 90\%$ identical (frame build, canonical rotation, `I_plus_local` routing, buffer gate, inline dispatch, `v_imp` recovery, RHS scatter). Extract that body into one routine parameterized by the few genuinely different things: where `Q_nbr` comes from (local vDofs vs face-neighbor vDofs) and whether the boss-broadcast applies.

This is the phase that directly implements the stated requirement — “they should share the same code as much as possible.” Phases 1–2 make it *safe*; this phase makes it *permanent*.

Files to Modify

- `dynamic/wave_operator.inl` — extract; both `ComputeADERFaceFluxRHS` and `ComputeADER-SharedFaceFluxRHS` call it. Same for the RK pair, if the extraction is clean.
- `dynamic/wave_operator.hpp` — declare the helper.

Detailed Requirements

1. Extract:

```
/// Computes ONE fault QP's canonical-frame imposed states. Identical for
/// interior and shared faces: same frame rule (!elem1_on_plus), same routing,
/// same sub-step-buffer gate, same inline fallback.
void ComputeFaultQPImposedStates_(const FaultBasisQPData &qpd,
                                   bool                elem1_on_plus,
                                   int                 dof_idx,
```

```

const real_t      *I_self,
const real_t      *I_nbr,
real_t           dt,
DOFData          &fdata,
real_t           *I_imp_plus,
real_t           *I_imp_minus) const;

```

2. Land this as a **pure refactor**: no numerical change. Prove it with a bit-exactness gate.
3. Do the extraction **after** Phases 1–2, never before — refactoring on top of a known frame bug would bake the bug into the shared routine.

Acceptance Criteria

- ☐ Every trace ($np=1$ and $np=2/4$) is **bit-identical** before vs after this phase.
- ☐ `grep -c 'should_negate_frame' wave_operator.inl` drops from 6 to ≤ 2 .
- ☐ The (INV) invariant is asserted inside the single routine, so it covers both face classes.

Dependencies

- Depends on: Phase 2 (and ideally 3). Required by: nothing.

1.12 Phase 6: Revalidate and regenerate multi-rank references

In one sentence: we measure exactly how much the multi-rank answers moved, and regenerate the reference traces with sign-off.

Goal

Quantify the change, confirm it is an improvement (convergence toward the $np=1$ answer), and refresh gold data.

Detailed Requirements

1. For TPV205 spatial, TPV102, TPV104: run $np \in \{1, 2, 4, 8\}$ before and after. Report $\max |X_{np} - X_1|$ for on-fault slip, slip-rate, and rupture time. **Expected:** the metric *decreases* after the change (np -independence improves). If it increases, STOP — the fix is wrong.
2. `symmirror_1000m` $np=10$ sweep, `ader_order` $\in \{2, 3, 4\}$, reproducing `ADER_ITERATOR_SYM1000_RESULTS.md`'s table. Expect substep dip-drift $\leq$ one-shot dip-drift, and no $1e28$.
3. **Frontera:** do not submit anything without explicit user approval (project memory).
4. Regenerate $np>1$ gold traces only after (1) and (2) pass and the user signs off.

Acceptance Criteria

- ☐ $\max |X_{np} - X_1|$ decreases for every problem and every np .
- ☐ No overflow on the y-mirror mesh at $np=10$ for any ADER order.
- ☐ User has signed off on regenerating $np>1$ gold traces.

Dependencies

- Depends on: Phases 1–5.

1.13 Testing Strategy

Phase	Test	What it proves
0	test_shared_fault_substep_parity (RED)	The <code>bp2</code> exists and is measurable.
0	(INV) check on a y-mirror fixture (RED)	The <i>cause</i> is the frame bit, not something else.
1	VerifySharedFaultCanonicalFrame + tamper test	The frame is now rank-independent and (INV) holds.
1	np=1 bit-exactness	The fix cannot touch serial results.
2	Phase-0 test flips to GREEN	Interior and shared now agree.
2	symmirror np=10, no overflow	R-1601's failure mode is genuinely gone.
3	Reconcile assertion + negative control	Divergence can never hide again.
4	Exact seam/interior μ equality	R-002 is closed, not bounded.
5	Bit-identical before/after	The refactor is pure.
6	np-convergence metric decreases	The change is an improvement, not just a change.

Correctness oracle. np=1 is the reference: it has no shared faces, so it is the *definition* of the un-decomposed answer. Every multi-rank result must converge toward it. The existing test_friction_substep_iterator_parity (36 bit-exact assertions) guards the serial iterator.

1.14 Risk Assessment

Risk	Severity	Mitigation	Related constraint
Re-enabling the sub-step buffer on shared faces brings back the 4e28 overflow that R-1601 was written to stop.	CRITICAL	Phase 1 lands the frame fix <i>and its cross-rank assertion</i> first; Phase 1's acceptance criterion runs the exact failing mesh (symmirror, np=10) with the fallback still in place, isolating frame from re-enable.	Do not blind-revert R-1601.
The diagnosis is wrong. qpd.normal already has its sign baked in and sign_flipped is documented “diagnostic only”, so the <i>meaning</i> of negating by either bit is not fully pinned. The interior and shared rules might both be correct under conventions this plan has not established.	CRITICAL	Phase 1 requirement 0 pins the contract and asserts it on interior faces first. Phase 1 requirement 1 ships a read-only census that must show zero bit-disagreement on non-mirror meshes and non-zero on symmirror. If either result contradicts the diagnosis, STOP and re-plan. No behavior changes until both pass.	Do not plan what you don't understand.
!elem1_on_plus does not actually give a rank-independent frame on shared faces, so the “one rule” still yields two different canonical frames across a seam.	CRITICAL	VerifySharedFaultCanonical before Phase 2 depends on it. Explicit STOP-and-re-plan gate.	Do not plan what you don't understand.
Multi-rank gold traces for TPV102/104/205/SAFS change, invalidating stored references.	HIGH	Phase 6 quantifies the move and requires it to be <i>toward</i> the np=1 answer. Regeneration needs user sign-off.	Multi-rank references will change.

Risk	Severity	Mitigation	Related constraint
The frame fix at :3646 also changes the RK path, which the original scope tried to leave alone.	MODERATE	The RK shared path has the <i>same</i> latent (INV) violation; leaving it is knowingly shipping a bug. Fix all three sites, and add RK np>1 to Phase 6's sweep. Flag explicitly for sign-off.	RK path "untouched" cannot be honored without leaving a defect.
The boss-broadcast is currently load-bearing for correctness, so demoting it to an assertion exposes latent divergence.	MODERATE	Phase 3 keeps the overwrite and only <i>adds</i> the check. If the assertion fires, that is a real bug being surfaced, not a regression introduced.	—
Local MPI is capped at $np \leq 10$, so np=10 symmmirror is the largest reproduction available.	LOW	Sufficient: the recorded failure is at np=10. Larger sweeps go to Frontera with approval.	No Frontera runs without approval.
Phase 0's np=2 fixture does not actually place fault faces on the seam (METIS decides).	LOW	Force the partition explicitly; assert GetNumSharedFaultQPs() > 0 or fail the fixture.	—

1.15 Appendix A – Evidence index

Claim	Anchor
Interior consumes the sub-step buffer Shared discards it and re-solves inline	wave_operator.inl:4288–4327 wave_operator.inl:5296–5342 ((void)substep_I_imp_plus_flat_;
Iterator advances all QPs incl. shared	friction_substep_iterator.hpp:227 (for i < dof_data.size()), :236–244
Frame bit: interior	:2221, :2972, :4217 — !elem1_on_plus
Frame bit: shared	:2513, :3646, :5252 — qpd.sign_flipped
Side-routing bit (both)	:2247–2249, :2614–2619, :4271–4273, :5278–5280 — elem1_on_plus
sign_flipped is FP-bimodal on y-mirror meshes	wave_operator.hpp:835 (R-101 rationale)

Claim	Anchor
sign_flipped is “diagnostic only; sign already baked into basis vectors”	fault/fault_basis.hpp:33 (and :44)
sign_flipped = NormalNeedsFlipToCanonical(n_raw, ref_normal, dim), per QP	fault/fault_basis.hpp:427
elem1_on_plus is per-face, from centroid projection onto the <i>canonicalized</i> normal	wave_operator.inl:435–447 (interior), :495–521 (shared)
Canonicalized normal is rank-independent; exactly one rank gets elem1_on_plus == true	wave_operator.inl:498–508 (comment)
R-1601 rationale + failure signature	wave_operator.inl:5285–5313; ADER_ITERATOR_SYM1000_RESULTS.md:23
symmirror is a y-mirror mesh	ADER_ITERATOR_SYM1000_RESULTS.md:10
Unified shared gate previously landed, then rolled back	SUBSTEP_ITERATOR_MPI_FIX.md §2 (R-1300/R-1304)
MPI parity tests never authored (“merge-blocking”)	SUBSTEP_ITERATOR_MPI_FIX.md, “What is NOT covered”
Neighbor Q available pointwise per sub-step	wave_operator.inl:2318–2330
Boss-broadcast reconcile	wave_operator.inl:5686–5749
Stale docstring	wave_operator.hpp:555–562
Antiparallel traction $\Rightarrow$ unbounded growth	CLAUDE.md, “Sign Conventions”